Exercise_1

Code:

```c
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char name[20];
    int grade;
    int class;
    float average;
}Student ;


int main() {
    Student *s1 = malloc(sizeof(Student));

    strcpy(s1->name, "고길동");
    s1->grade = 1;
    s1->class = 3;
    s1->average = 65.389999f;

    printf("이름: %s\n", s1->name);
    printf("학년: %d\n", s1->grade);
    printf("반: %d\n", s1->class);
    printf("평균점수: %f\n", s1->average);

    free(s1);

    return 0;
}
```

Result:



Exercise_2

Code:

```c
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char building_name[100];
    int room_number;
    char room_name[100];
}Handong;

int main() {
    Handong room1, room2, room3;

    Handong* room1Pt = &room1;
    Handong* room2Pt = &room2;
    Handong* room3Pt = &room3;


    strcpy(room1.building_name, "Newton");
    room1.room_number   = 109;
    strcpy(room1.room_name, "iiLAB");

    strcpy(room2.building_name, "Newton");
    room2.room_number = 118;
    strcpy(room2.room_name, "Control-Lab");

    strcpy(room3.building_name, "Newton");
    room3.room_number = 119;
    strcpy(room3.room_name, "SW-Lab");

    // using structure
    printf("1. using structure\n");
    printf("%s building, room %4d is %s\n", room1.building_name, room1.room_number, room1.room_name);
    printf("%s building, room %4d is %s\n", room2.building_name, room2.room_number, room2.room_name);
    printf("%s building, room %4d is %s\n", room3.building_name, room3.room_number, room3.room_name);

    printf("\n\n");

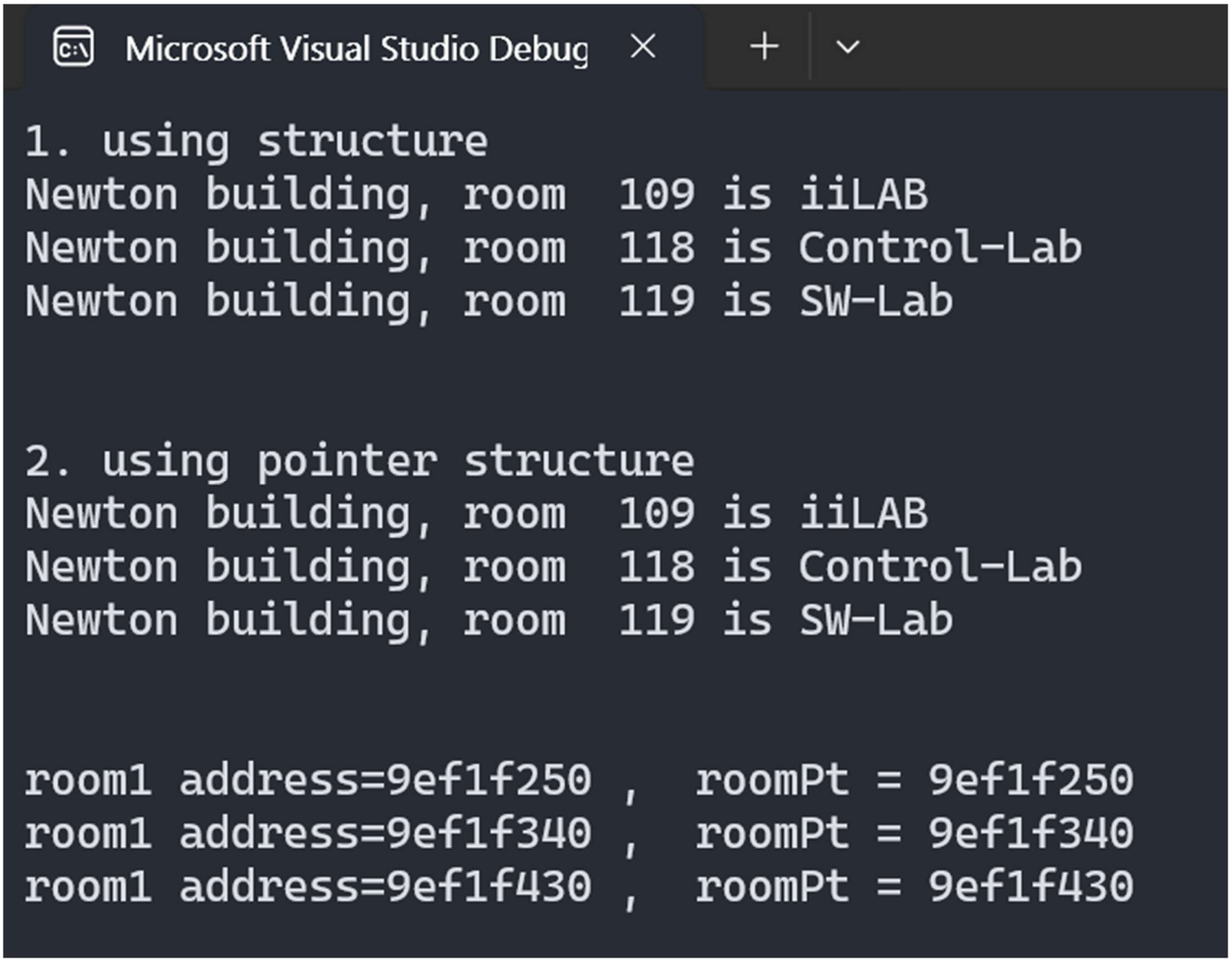
    // using pointer structure
    printf("2. using pointer structure\n");
    printf("%s building, room %4d is %s\n", room1Pt->building_name, room1Pt->room_number, room1Pt->room_name);
    printf("%s building, room %4d is %s\n", room2Pt->building_name, room2Pt->room_number, room2Pt->room_name);
    printf("%s building, room %4d is %s\n", room3Pt->building_name, room3Pt->room_number, room3Pt->room_name);

    printf("\n\n");

    printf("room1 address=%x ,  roomPt = %x\n", (unsigned)&room1, (unsigned)room1Pt);
    printf("room1 address=%x ,  roomPt = %x\n", (unsigned)&room2, (unsigned)room2Pt);
    printf("room1 address=%x ,  roomPt = %x\n", (unsigned)&room3, (unsigned)room3Pt);

    return 0;
}
```

Result:


Microsoft Visual Studio Debug
1. using structure
Newton building, room 109 is iiLAB
Newton building, room 118 is Control-Lab
Newton building, room 119 is SW-Lab
2. using pointer structure
Newton building, room 109 is iiLAB
Newton building, room 118 is Control-Lab
Newton building, room 119 is SW-Lab
room1 address=9ef1f250 , roomPt = 9ef1f250
room1 address=9ef1f340 , roomPt = 9ef1f340
room1 address=9ef1f430 , roomPt = 9ef1f430

Exercise_3:

Code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>

typedef struct {
    int x;
    int y;
    int z;
} POSITION_TypeDef;

//
void addPos(POSITION_TypeDef pos0, POSITION_TypeDef pos1, POSITION_TypeDef* posOut);
void getDist(POSITION_TypeDef pos0, POSITION_TypeDef pos1, POSITION_TypeDef* posOut);
void printPos(POSITION_TypeDef Pos);

void addPos(POSITION_TypeDef pos0, POSITION_TypeDef pos1, POSITION_TypeDef* posOut) {
    posOut->x = pos0.x + pos1.x;
    posOut->y = pos0.y + pos1.y;
    posOut->z = pos0.z + pos1.z;
}

void getDist(POSITION_TypeDef pos0, POSITION_TypeDef pos1, POSITION_TypeDef* posOut) {
    posOut->x = abs(pos0.x - pos1.x);
    posOut->y = abs(pos0.y - pos1.y);
    posOut->z = abs(pos0.z - pos1.z);
}

void printPos(POSITION_TypeDef Pos) {
    printf("PosOut result : (x, y, z) = (%d, %d, %d)\n", Pos.x, Pos.y, Pos.z);
}

void main()
{
    // Exercise 3 *****************************************
    printf("\nExercise 3\n");
    POSITION_TypeDef Pos[2];
    POSITION_TypeDef PosOut = { 0, };

    Pos[0].x = 2; Pos[0].y = 2; Pos[0].z = 2;
    Pos[1].x = 5; Pos[1].y = 5; Pos[1].z = 5;

    printf("Result output of 'addPos'\n");
    addPos(Pos[0], Pos[1], &PosOut);
    printPos(PosOut);

    printf("Result output of 'getDist'\n");
    getDist(Pos[0], Pos[1], &PosOut);
    printPos(PosOut);

    system("pause");
}
```

Result:

```
Exercise 3
Result output of 'addPos'
PosOut result : (x, y, z) = (7, 7, 7)
Result output of 'getDist'
PosOut result : (x, y, z) = (3, 3, 3)
Press any key to continue . . .
```